# Python Data Engines: State of the Union 2026

Every Python data engine looks impressive when you throw a 64-core machine at it. That is not this talk.

# Agenda

01 What changed since 2025

02 Iceberg

03 The engines

04 What does “fast” mean?

05 Benchmarks

06 Real-life use case

07 What I learned

# What changed since 2025

**SQL**

Polars SQL went from a side feature to all 99 TPC-DS-like queries.

Given enough time, every DataFrame library becomes a database.

**Iceberg**

Iceberg is the new baseline.

Last year most of these engines couldn’t write a table; now they all write Iceberg.

**AI**

I don’t care about the lack of a standard DataFrame API.

I became a dbt believer only because of AI: I was scared of Jinja.

I don’t write any code, I just review the tests.

**Python engines aren’t niche anymore. They’re mainstream, and a bit boring.**

02

# Iceberg

All engines are equal, as long as they speak the same spec.

# How we got here

## 1990s

### The warehouse

Storage and compute in one product. The engine owns the data; changing engine is a migration.

## 2006

### Big data: Hadoop

Files spread over a cluster's disks. The data is open, but you need the cluster to read it.

## 2010s

### The cloud warehouse

Elastic compute, but the tables live inside one vendor.

## 2017

### Open table formats

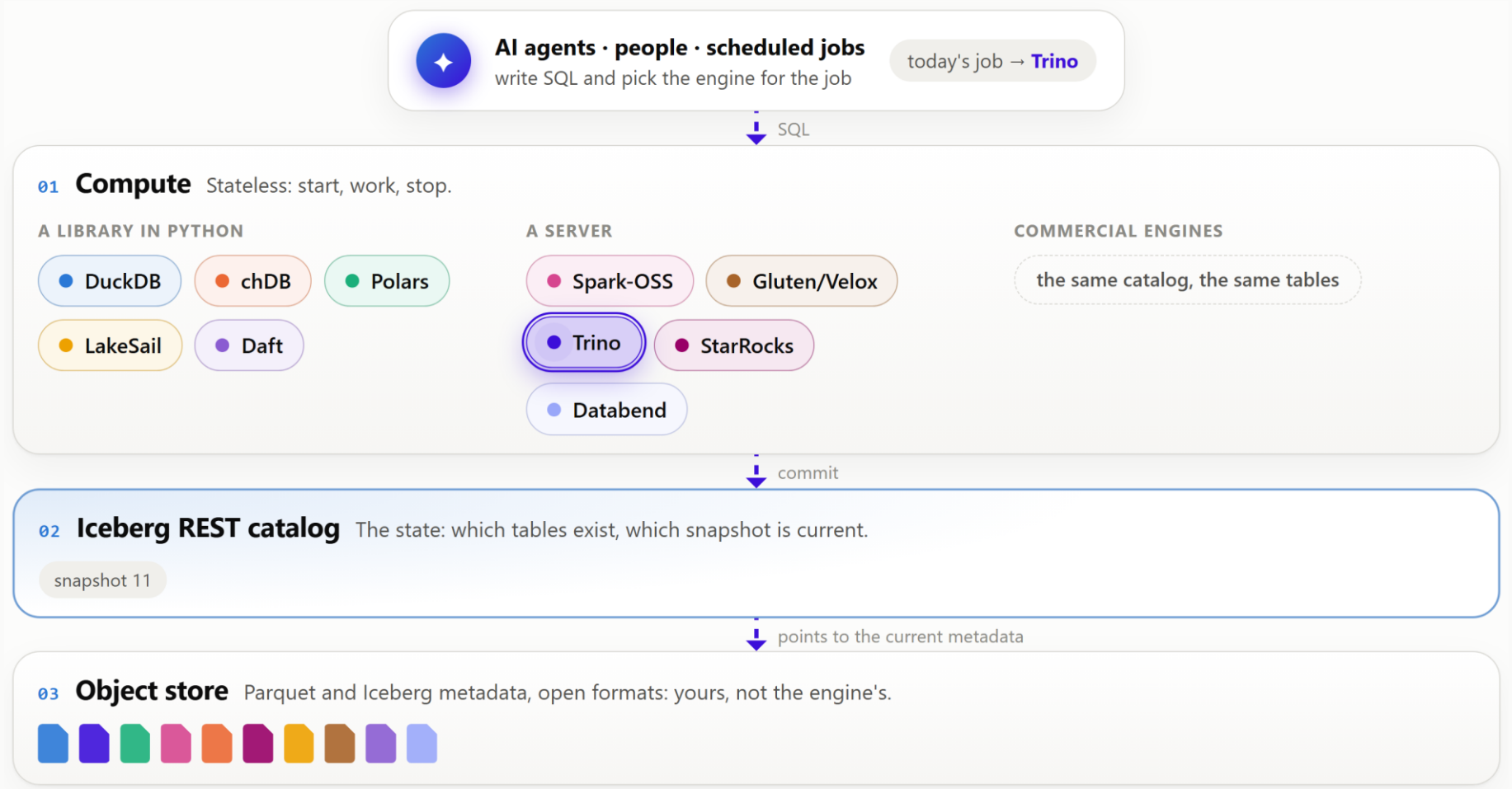
Iceberg (Netflix), Hudi, Delta: tables in object storage, but writing one needed a Java cluster (Spark, Trino).

## Now

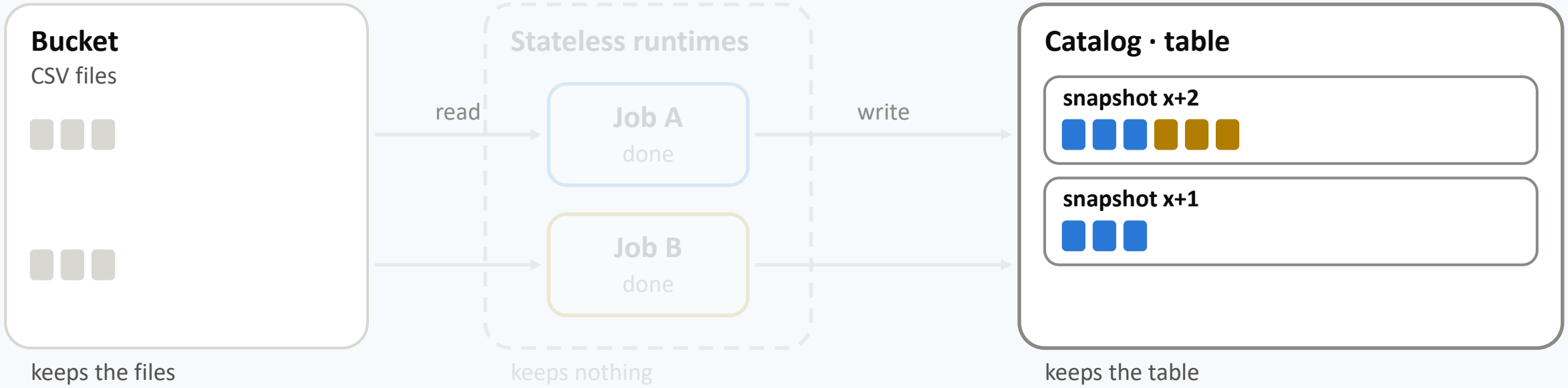
### The open lakehouse

A REST catalog, and table libraries in Rust, C++ and Python: a library in a Python process writes the same tables as a cluster.

# The open lakehouse



# The catalog is the referee




Both jobs stop. Snapshot x+2 has the files of both jobs. If both jobs had changed the same files, the retry would fail.

03

# The engines

16 engines tried, one catalog, one small box.

# Disclosure: what this runs on

Compute

## Free GitHub runners

**4 cores · 16 GB**

Catalog

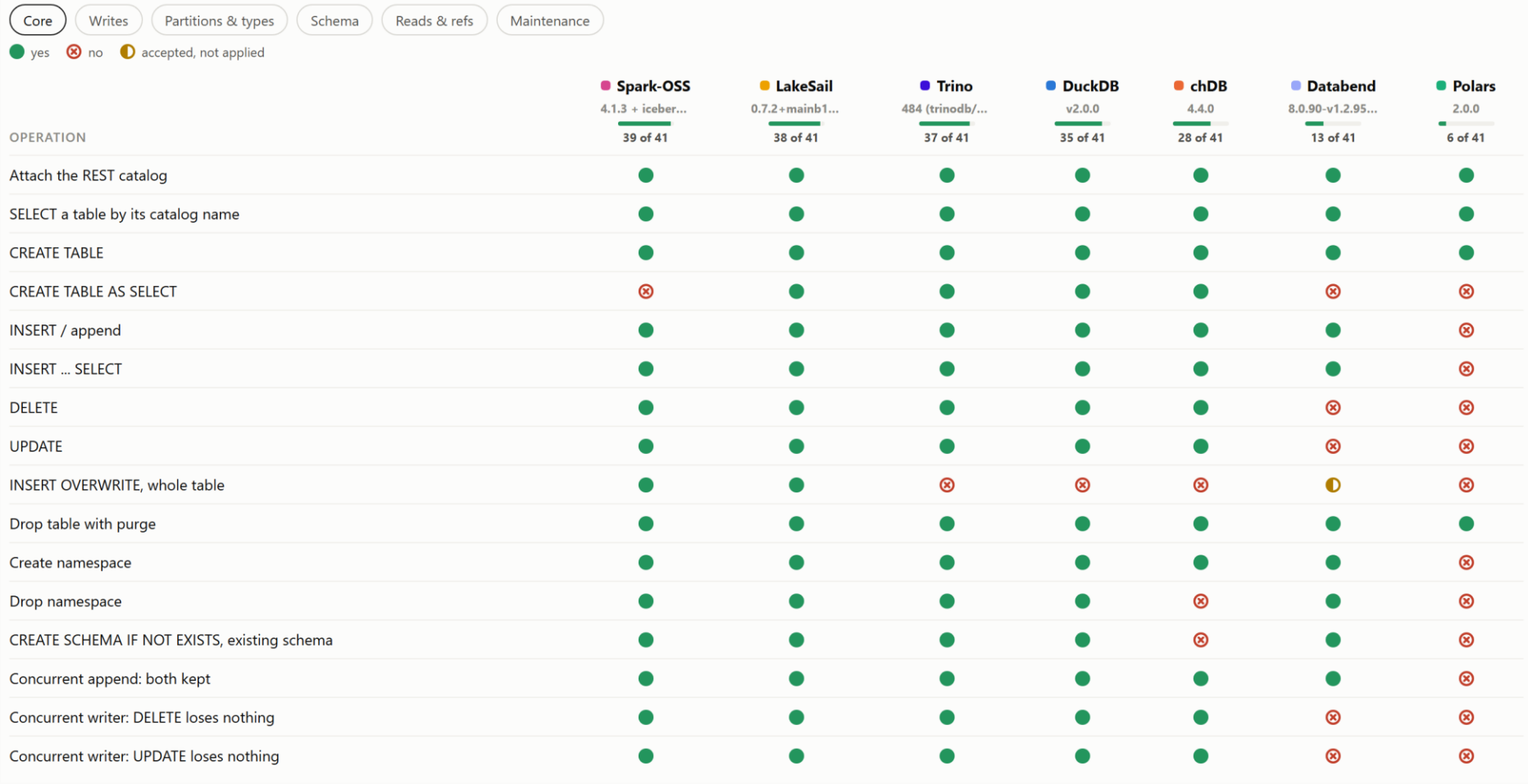
## OneLake Catalog

Free through my job :)

# Who they are

| ENGINE | STARTED | WRITTEN IN | RUNS AS |
|---|---|---|---|
| Spark-OSS | UC Berkeley AMPLab, 2009 | Scala | Python drives a local JVM |
| Gluten/Velox | Intel and Kyligence, 2022; Velox by Meta | C++, Scala | Python drives a local JVM |
| Trino | Facebook as Presto, 2012; Trino since 2020 | Java | Client-server |
| StarRocks | Fork of Apache Doris, 2020 | C++, Java | Client-server |
| Databend | Datafuse Labs, 2020 | Rust | Client-server |
| DuckDB | CWI Amsterdam, 2019 | C++ | In-process |
| chDB | Auxten Wang, 2023 | C++ | In-process |
| Polars | Ritchie Vink, 2020 | Rust | In-process |
| LakeSail | LakeSail, 2024 | Rust | Client-server, Spark Connect |
| Daft | Eventual, 2022 | Rust | In-process |
| Pandas | Wes McKinney, 2008 | Python, C | In-process |

# The Iceberg support grid



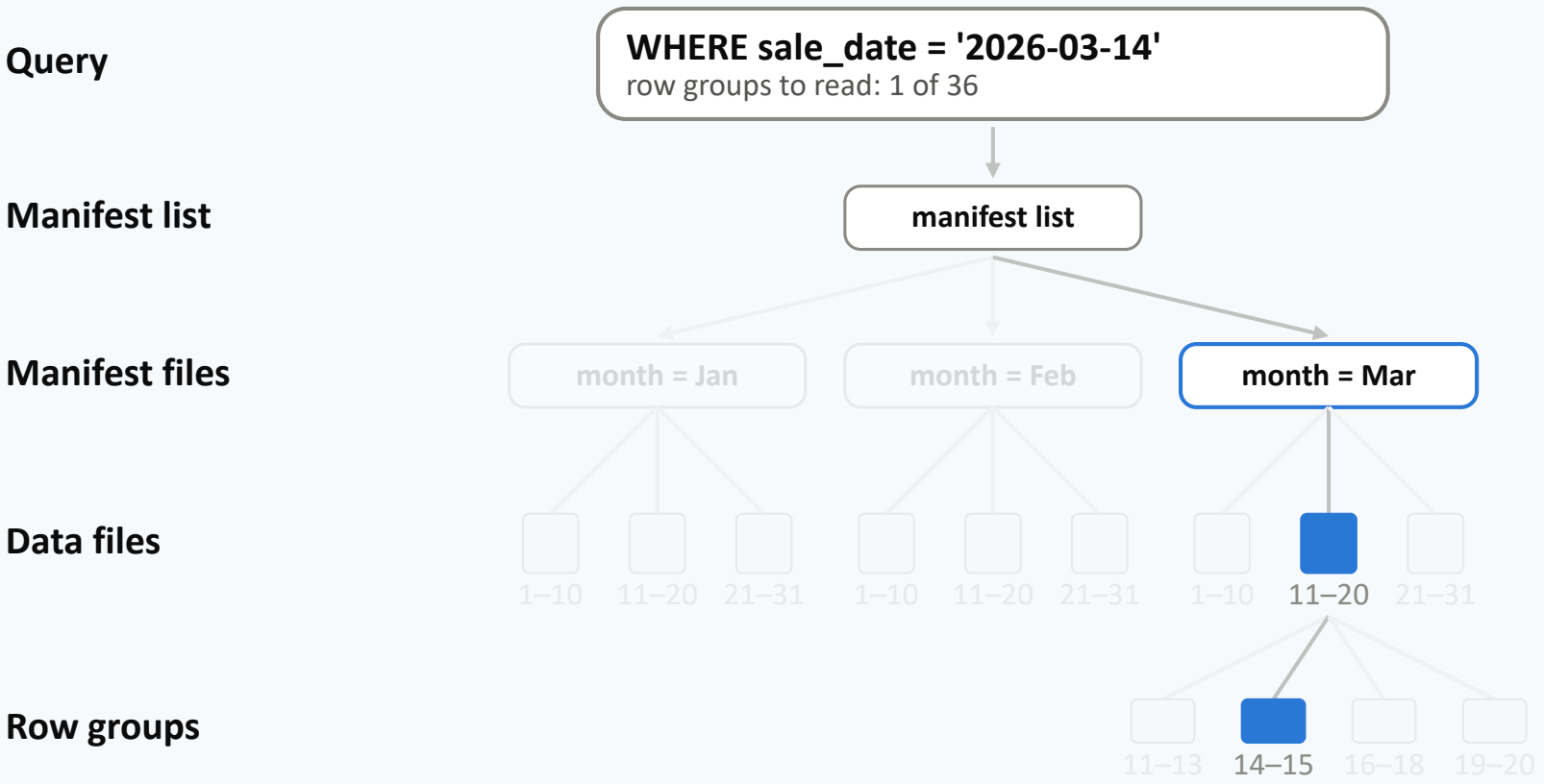

| OPERATION | Spark-OSS 4.1.3 + iceber... 39 of 41 | LakeSail 0.7.2+mainb1... 38 of 41 | Trino 484 (trinodb/... 37 of 41 | DuckDB v2.0.0 35 of 41 | chDB 4.4.0 28 of 41 | Databend 8.0.90-v1.2.95... 13 of 41 | Polars 2.0.0 6 of 41 |
|---|---|---|---|---|---|---|---|
| Attach the REST catalog | ● | ● | ● | ● | ● | ● | ● |
| SELECT a table by its catalog name | ● | ● | ● | ● | ● | ● | ● |
| CREATE TABLE | ● | ● | ● | ● | ● | ● | ● |
| CREATE TABLE AS SELECT | ⊗ | ● | ● | ● | ● | ⊗ | ⊗ |
| INSERT / append | ● | ● | ● | ● | ● | ● | ⊗ |
| INSERT ... SELECT | ● | ● | ● | ● | ● | ● | ⊗ |
| DELETE | ● | ● | ● | ● | ● | ⊗ | ⊗ |
| UPDATE | ● | ● | ● | ● | ● | ⊗ | ⊗ |
| INSERT OVERWRITE, whole table | ● | ● | ⊗ | ⊗ | ⊗ | ◐ | ⊗ |
| Drop table with purge | ● | ● | ● | ● | ● | ● | ● |
| Create namespace | ● | ● | ● | ● | ● | ● | ⊗ |
| Drop namespace | ● | ● | ● | ● | ⊗ | ● | ⊗ |
| CREATE SCHEMA IF NOT EXISTS, existing schema | ● | ● | ● | ● | ⊗ | ● | ⊗ |
| Concurrent append: both kept | ● | ● | ● | ● | ● | ● | ⊗ |
| Concurrent writer: DELETE loses nothing | ● | ● | ● | ● | ● | ⊗ | ⊗ |
| Concurrent writer: UPDATE loses nothing | ● | ● | ● | ● | ● | ⊗ | ⊗ |

04

# What does “fast” mean?

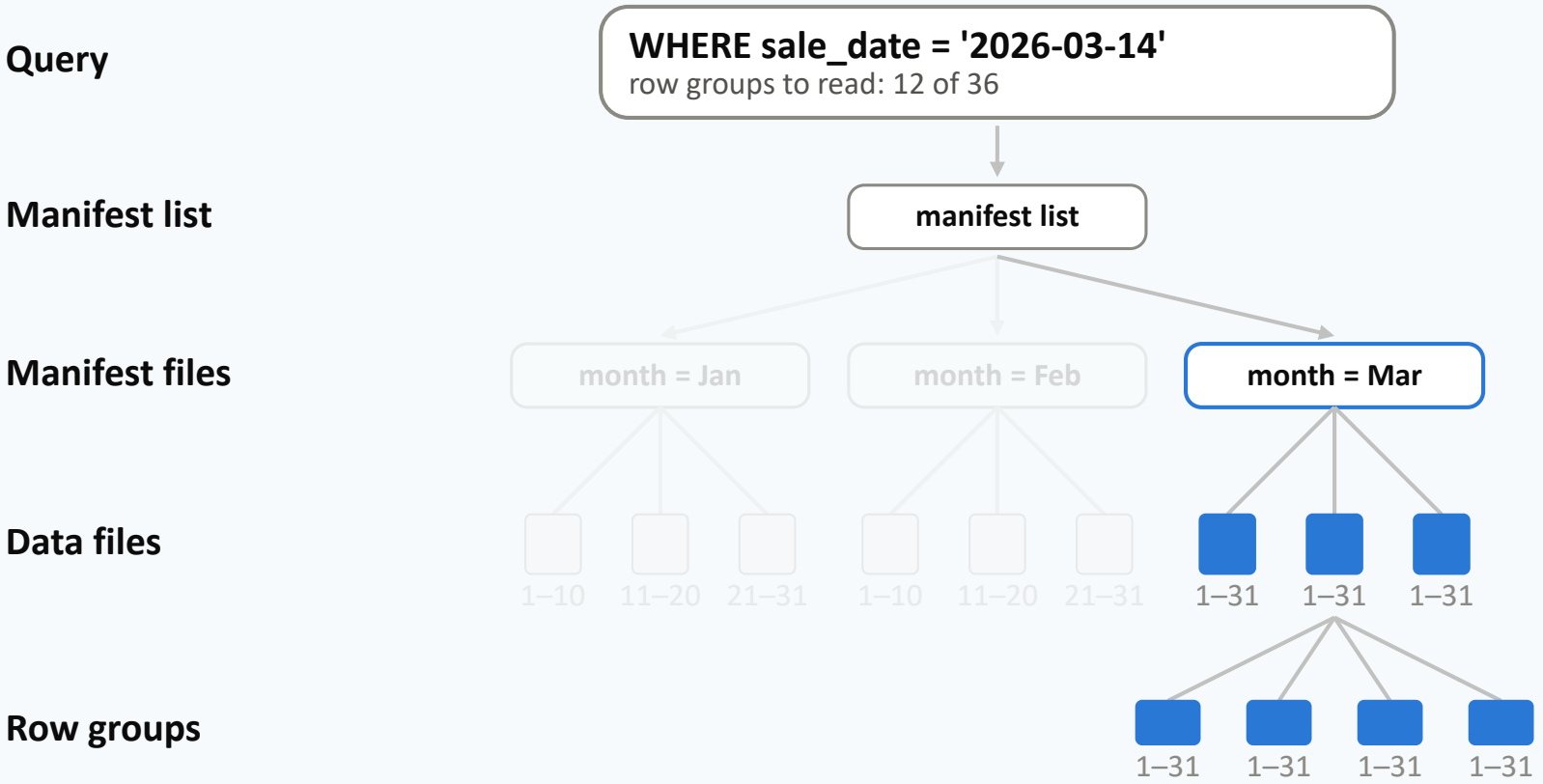
The best query is no query.

# File skipping




1 row group of 36, before a single row was read.

# The importance of sorting data




Random order: every March file spans days 1–31, so 12 row groups instead of 1.

# The small-file problem

The same 1 GB of data, written two ways.

**8 files of 128 MB**

**8 opens, 8 footers,**
**8 manifest entries.**

**8,000 files of 128 KB**

**8,000 opens, 8,000 footers,**
**8,000 manifest entries.**

Every file costs a request to storage before a single row is read.

**Compact into a few big files. Iceberg’s default target is 512 MB.**

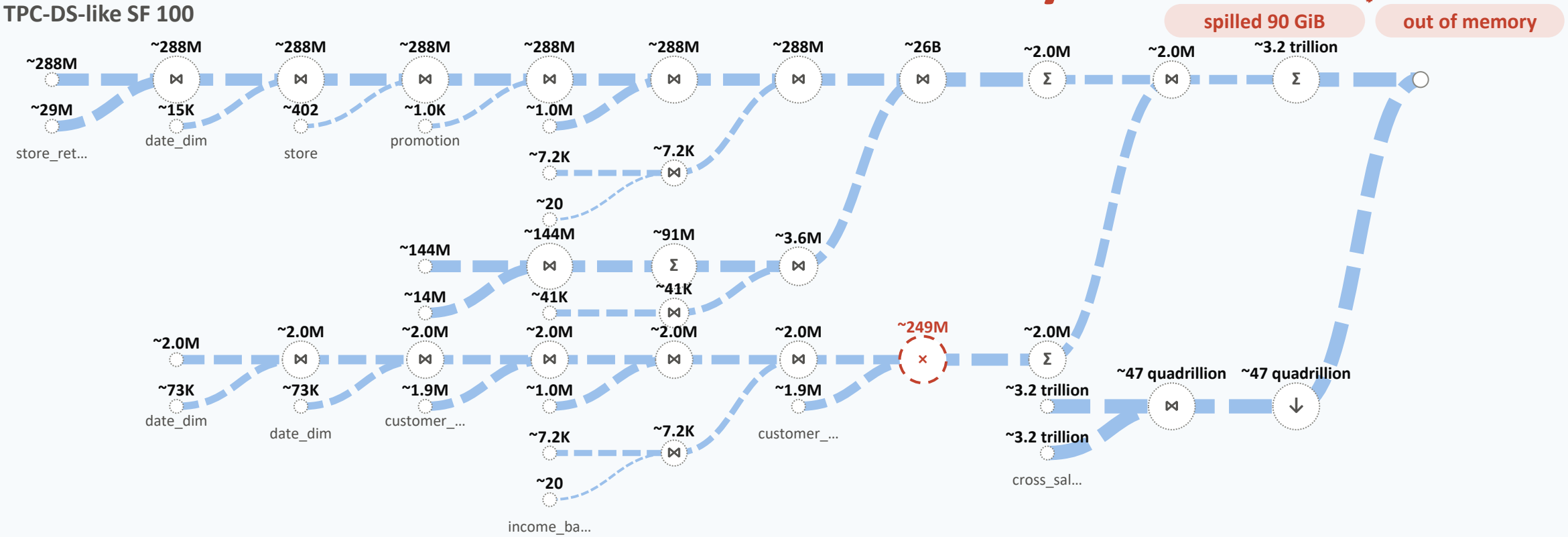
# Q64 on DuckDB 2.0: a bad guess

**DuckDB 2.0**

TPC-DS-like SF 100

**Out of memory**

✗ 247 s




DuckDB 2.0 guesses a ≠ join makes 249M rows and runs it early, on its own. It makes about 3 trillion.

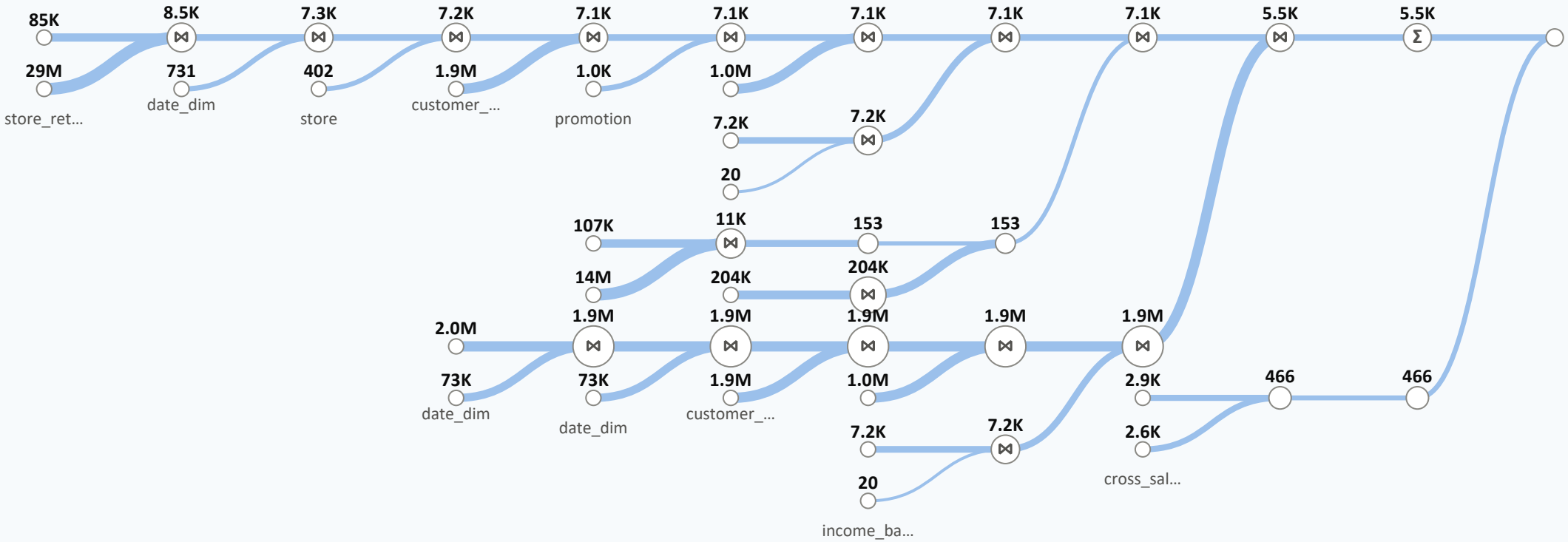
# Q64 on the DuckDB nightly: fixed upstream

## DuckDB nightly

✓ 101 s

done

**TPC-DS-like SF 100** · peak memory **8.0 GiB** · largest intermediate result **1.9M** rows




duckdb#27493 prices ≠ as nearly a cross product, so it is checked inside a hash join. Done in 101 s, no step above 1.9M rows.

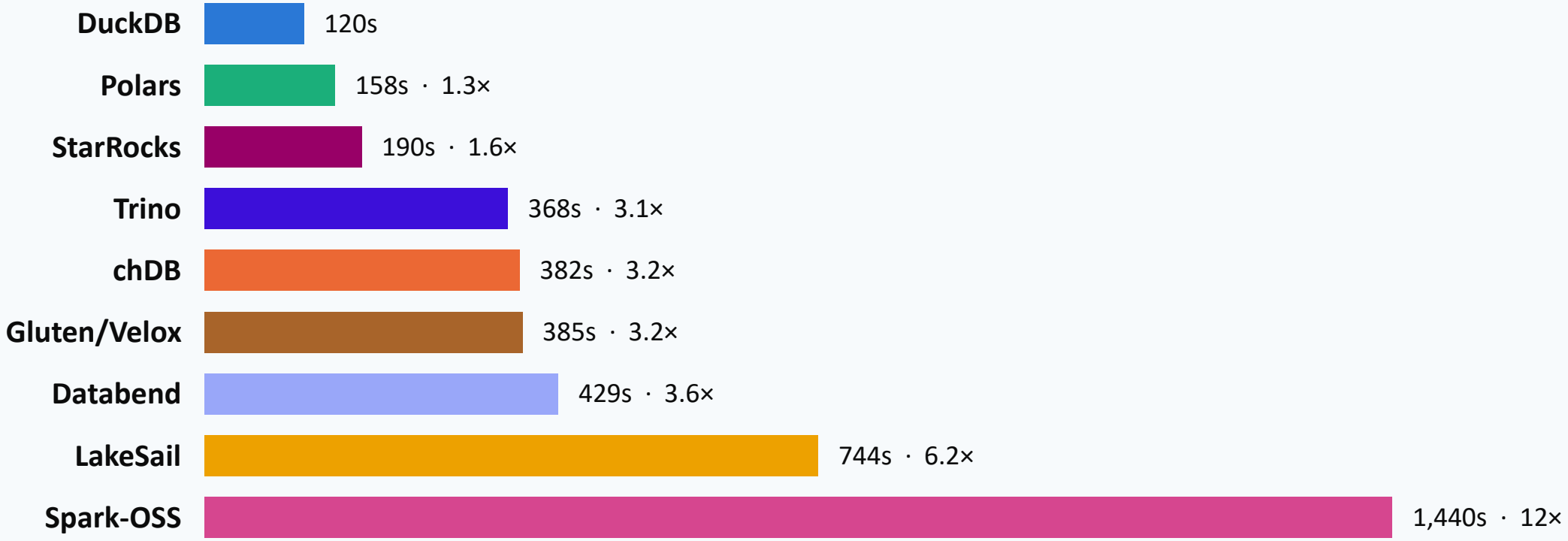
05

# Benchmarks

Nobody switches engines for 2×.

# TPC-H-like, scale factor 30: all nine engines finish

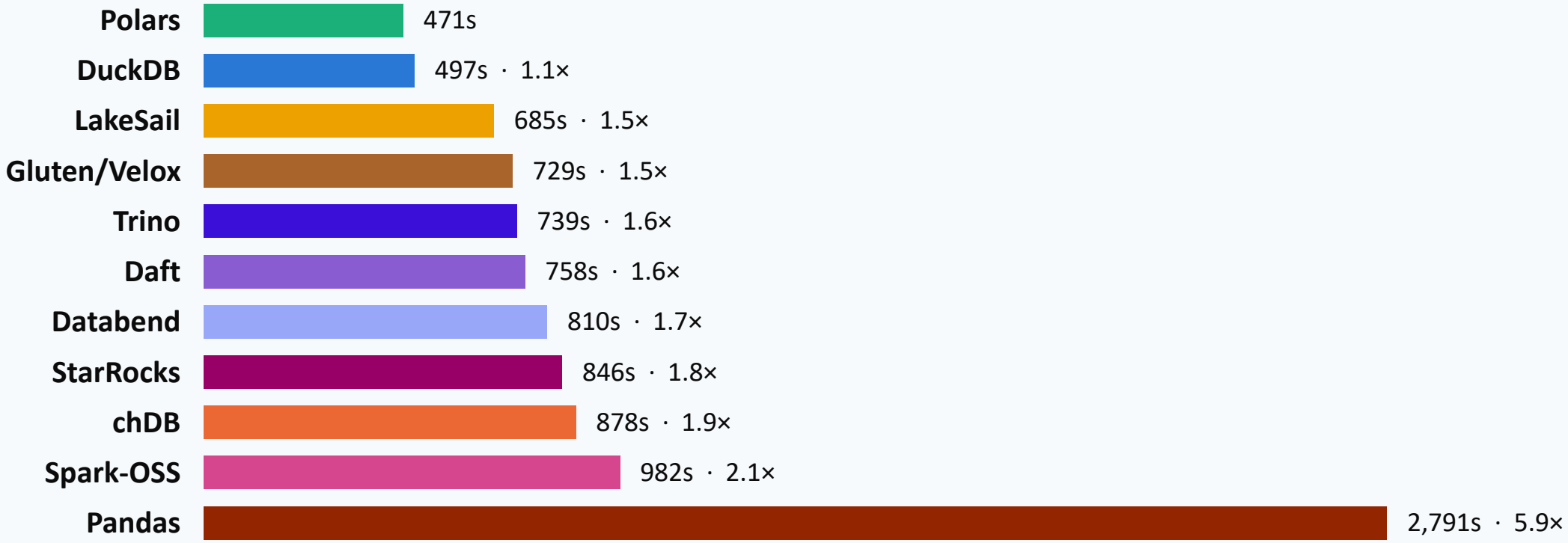
Total cold seconds for all 22 queries at scale factor 30.




Derived from TPC-H, not audited. It tests the cache more than anything else.

# Light ETL: 1,000 CSV files into one Iceberg table

Read 1,000 CSVs and write one Iceberg table.

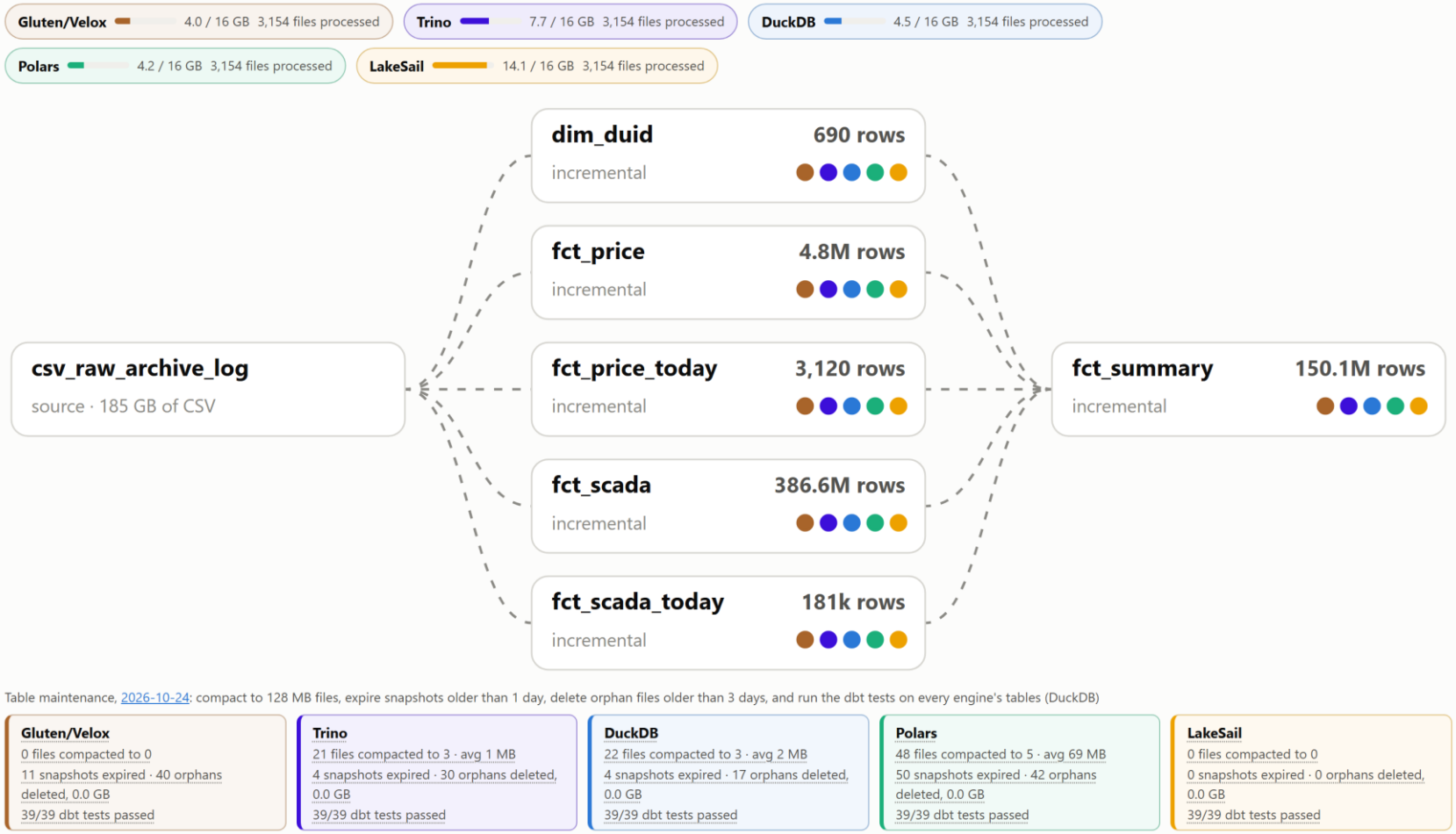


06

# Real-life use case

The only one that matters.

# The real workload



# Why real life is hard

### A minor feature can be a blocker

A ragged CSV: 53 fields on one row, about 120 on the next. If the reader can’t take it, nothing runs.

### Append only works in a demo

Runs overlap. Two appends of the same files both commit, and the rows land twice. A MERGE on the key keeps one copy.

### Spill to disk is the real bottleneck

The merge doesn’t fit in 16 GB of RAM, so it goes to disk: up to 60 GB in one run. No spill, and the runner dies.

### The adapter matters as much as the engine

dbt decides how the batch is staged: a temp table or a view. Same SQL, very different memory.

**Every one of these can stop an engine, whatever its benchmark says.**

# Every report, one dot each

The best help for an engine is a bug report it can reproduce. No PR needed: the maintainers fix it, often within days.

**97** bug reports filed upstream

**23** fixed

**3 days** median, from report to fix

**4** pull requests from the bench




**What makes a report land:** one script on a clean machine · the smallest case · expected and actual · the version, and main · one bug per issue (Daft #7532 became #7601 and #7602).

*AI drafts the repro, I check it, one bug per issue.*

07

# What I learned

# Read less

On a small box, bytes off storage cost more than CPU.

**Cache your reads**

## 370 s → 92 s

DuckDB, TPC-DS-like SF10 cold: file cache off, then on.

**Filter on a date, not a file name**

## failed → 168 s

Sail's merge. Iceberg cuts string bounds to 16 characters; dates are never cut.

# Distributed engines run fine on one small box

Each on one 4-core, 16 GB runner.

**StarRocks · TPC-H-like SF300**

## 3,943s

A Java front end and a C++ back end, sharing one VM.

**Spark, Velox underneath · TPC-DS-like SF100**

## 5,762s

3.7× faster than Spark’s own engine, same SQL. Never ran out of memory here.

# Engine execution nuances

One value, 96.07385, cast from DOUBLE to DECIMAL(18,4)

**Spark**

**96.0739**

**DuckDB**

**96.0738**

Out of 3.08M rows: DuckDB differs from Spark on 6,716, Polars on 85,090. Each by 0.0001.

**Same SQL, not the same behaviour. The fix: DECIMAL(18,5), nothing rounds.**

# Instead of flooding projects with AI pull requests, report bugs.

**Bugs, not feature requests.**

**Check it isn’t a duplicate.**

**The smallest possible repro.**

**These engines work, and that’s the story.**







# Thank you

Questions? · djouallah.github.io/lakehouse_benchmark

Scan for the code:
github.com/djouallah/lakehouse_benchmark